M11 Hooks : Événements & Système

Méthodologie Intermediate

Réagir automatiquement aux actions de Claude Code

Qu'est-ce qu'un hook

Les hooks de type commande s'exécutent sur les événements configurés et reçoivent du JSON sur stdin. Il existe aussi des hooks prompt, agent, HTTP et MCP tool ; leur disponibilité dépend de l'événement. Claude Code compte 33 événements de hook au total ; les tableaux montrent les principaux.

Événements principaux

Événement	Moment	Peut bloquer
`SessionStart`	Démarrage ou reprise	Non
`SessionEnd`	Fin de session	Non
`UserPromptSubmit`	Avant traitement du prompt	Oui
`PreToolUse`	Avant l'appel d'un outil	Oui
`PostToolUse`	Après succès d'un outil	Non
`PreCompact`	Avant compaction	Oui
`Notification`	Notification	Non
`Stop`	Claude finit de répondre	Oui
`SubagentStop`	Fin d'un sous-agent	Oui

Événements multi-agents et configuration

Événement	Depuis	Utilité
`TeammateIdle`	v2.1.32+	Un membre va devenir inactif
`TaskCompleted`	v2.1.32+	Une tâche va être terminée
`ConfigChange`	v2.1.49+	La configuration change

Les contrôles d'équipe peuvent demander du travail supplémentaire avant la clôture. `ConfigChange` audite la configuration indépendamment d'Agent Teams.

Configuration dans settings.json

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/
security.sh",
        "args": [], "timeout": 10
      }]
    }]
  }
}
```

Champs de configuration

Champ	Usage
`command`	Exécutable si `args` est présent ; sinon commande shell
`args`	Arguments sans shell ; ne remplace pas l'exécutable
`timeout`	Secondes ; défauts : command 600, prompt 30, agent 60
`async`	Si `true`, continuer sans attendre
`continueOnBlock`	Hooks prompt : continuer après `ok: false` sur `PreToolUse`, `PostToolUse` et `TeammateIdle`
`asyncRewake`	S'exécute en arrière-plan et réveille Claude sur le code de sortie 2 (implique `async`)
`onFailure`	`"block"` bloque l'action si un hook command ou HTTP ne démarre pas, expire ou se termine de façon inattendue (v2.1.295 ; défaut `continue`)

Codes de sortie

Code	Signification
`0`	Succès ; un JSON valide peut encore refuser l'action
`2`	Blocage si l'événement le permet ; effet variable
Autre non-0	Généralement non bloquant, sans JSON de décision

Pour `PreToolUse`, exit 2 empêche l'exécution. Le message stderr est transmis à Claude.

Synchrone vs Asynchrone

Garder la validation synchrone. `async: true` exécute un hook commande en arrière-plan, sans pouvoir bloquer l'action : notifications ou logs. Un formatage en arrière-plan peut entrer en concurrence avec une édition.

Flux des données stdin

```
{
  "tool_name": "Edit",
  "tool_input": { "file_path": "src/auth.ts" },
  "tool_response": { "success": true },
  "hook_event_name": "PostToolUse"
}
```

`Stop` inclut aussi `last_assistant_message`. Décisions et exceptions : référence des hooks.